
Modül 11 — Kafka ile Mesajlaşma

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — İdempotent Stok Güncelleyici (Kolay)	2
Ödev 2 — Zehirli Mesajları Dead Letter Topic'e Göndermek (Orta)	3
Ödev 3 — Outbox Relay (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/11-messaging-kafka/exercise/` altındadır. TODO yorumlarını bulun.
2. Testler gerçek bir Kafka broker'ına ve PostgreSQL'e (Testcontainers) karşı çalışır. **Docker açık olmalıdır.**
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/11-messaging-kafka/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/11-messaging-kafka/solution/` altındaki çözümü okuyun.

★ İpucu

Her ödevin kendi topic'leri vardır (ex1-orders, ex2-orders, ex3-orders) ve her test yeni, rastgele id'ler kullanır. Ödevler birbirini etkilemez.

Ödev 1 — İdempotent Stok Güncelleyici (Kolay)

Hedef: Bir consumer her `OrderPlaced` event'i için stoğu düşürür. Kafka en az bir kez teslim eder, yani aynı event iki kez gelebilir. Yine de yalnızca bir kez sayılmalıdır.

Inventory hazır verilmiştir: Kitap başına stoğu ve işlenmiş sipariş id'lerinin kümesini tutar.

Yapılacaklar (`exercisel.StockUpdater`):

- TODO 1a — TOPIC'i stock consumer group'u olarak dinleyin.
- TODO 1b — Her siparişi yalnızca bir kez uygulayın. `inventory.markProcessed(orderId)`, sipariş daha önce görüldüyse false döndürür.
- TODO 1c — Sipariş edilen kitabın stoğunu düşürün.

İpuçları:

- Ders bölüm 3.2: `@KafkaListener(topics = ..., groupId = ...)`.
- Stoğu değiştirmeden **önce** siparişi işlendi olarak işaretleyin, zaten işlendiyse durun.
- Gerçek bir uygulamada işlenmiş id'ler veritabanında, stok değişikliğiyle aynı transaction içinde saklanır.

Kabul kriterleri: `Exercise1Test` içindeki iki test de geçer.

Tahmini süre: 20 dakika

Ödev 2 — Zehirli Mesajları Dead Letter Topic'e Göndermek (Orta)

Hedef: Miktarı sıfır veya daha az olan bir sipariş asla ödenemez, JSON olmayan bir mesaj da asla okunamaz. İkisi de `ex2-orders.DLT`'ye gitmeli ve arkalarındaki mesajlar yine işlenmelidir.

`PaymentListener` ve `DeadLetterTemplates` hazır verilmiştir. Ayarlar zaten bir `ErrorHandlingDeserializer` kullanır.

Yapılacaklar (`exercise2.ErrorHandlingConfiguration`):

- TODO 2a — Bir `DefaultErrorHandler` bean'i. Boot onu her `@KafkaListener`'a ekler.
- TODO 2b — `Recoverer`'ı başarısız kayıtları `<orijinal topic>.DLT`'ye yayınlasın. `DeadLetterTemplates.create(...)` kullanın.
- TODO 2c — 100 ms arayla iki kez yeniden denesin.
- TODO 2d — Bir `InvalidQuantityException` asla yeniden denenmesin.

İpuçları:

- Ders bölüm 3.3 tüm yapılandırmayı gösterir.
- Hedef çözümleyici başarısız kaydı alır: `(record, exception) -> new TopicPartition(record.topic() + ".DLT", -1)`.
- `new FixedBackOff(100, 2)` ve `handler.addNotRetryableExceptions(...)`.

Kabul kriterleri: `Exercise2Test` içindeki 3 testin hepsi geçer. Sizin handler'ınız olmadan Boot'un varsayılan handler'ı 9 kez yeniden dener ve kaydı yalnızca loglar, bu yüzden DLT testleri başarısız olur.

Tahmini süre: 30 dakika

Ödev 3 — Outbox Relay (Zor)

Hedef: `OrderDesk`, `OrderPlaced` event'lerini outbox tablosunda saklar (hazır). Bunları Kafka'ya gönderen relay'i yazın: her event bir kez, yazıldığı sırayla, asla iki kez değil.

Yapılacaklar (`exercise3.OutboxRelay.relayBatch`):

- TODO 3a — Tek bir transaction içinde en fazla `batchSize` gönderilmemiş satırı en eskisinden başlayarak okuyun ve ikinci bir relay atlasın diye kilitleyin.
- TODO 3b — Her payload'ı `OrderPlaced` olarak, key'i `aggregate_id` olacak şekilde TOPIC'e gönderin ve broker'ın onayını bekleyin.
- TODO 3c — Gönderilen her satırı `sent_at = now()` ile işaretleyin.
- TODO 3d — Kaç satır gönderildiğini döndürün.

İpuçları:

- Ders bölüm 3.6: `SELECT ... WHERE sent_at IS NULL ORDER BY id LIMIT :limit FOR UPDATE SKIP LOCKED`.
- Metottaki `@Transactional`, satırlar işaretlenene kadar kilidi tutar.
- `json.readValue(payload, OrderPlaced.class)`, ardından `kafka.send(...).get(10, TimeUnit.SECONDS)`.
- `PendingEvent(long id, String aggregateId, String payload)` record'u, `JdbcClient`'in satırları eşlemesini sağlar (`aggregate_id` → `aggregateId`).

Kabul kriterleri: Exercise3Test içindeki 3 testin hepsi geçer.

Tahmini süre: 45 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Relay'iniz bir event'i gönderir ve sonra işaretler. Uygulama tam bu iki adımın arasında çökerse ne olur? Ödev 1'deki hangi bileşen sistemi bunun sonucundan korur? Sonra relay'i `@Scheduled` ile her 500 ms'de bir çalıştırın ve uygulamanın iki örneğini başlatın: `SKIP LOCKED` şimdi neden önemlidir?